LOGISTREAM

Sistema de procesamiento de eventos en tiempo real

Documentación técnica y guía de ejecución

Rocio Belen Giménez Cuenca

Dato	Valor
Proyecto	LogiStream – FPUNA
Tecnologías	Apache Kafka · Apache Beam · Python · Docker Compose
Apache Beam	2.69.0
Apache Kafka	4.0.1
Fecha	30 de septiembre / 1 de octubre de 2026

1. Descripción general

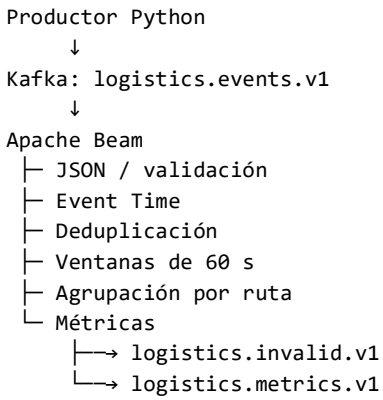
LogiStream es una solución de procesamiento de datos en streaming orientada al dominio de logística. Recibe eventos desde Apache Kafka, los procesa mediante Apache Beam y publica resultados agregados nuevamente en Kafka.

El pipeline contempla contrato de eventos, validación, tiempo de evento, deduplicación, ventanas temporales, manejo de eventos fuera de orden y tardíos, agregación por ruta y publicación de métricas.

1.1 Objetivo

Construir un pipeline reproducible de streaming capaz de transformar eventos logísticos en métricas por ruta y ventana temporal, manteniendo una separación entre eventos válidos e inválidos.

1.2 Flujo



1.3 Componentes

- Productor Python: genera eventos de prueba.
- Kafka: broker y transporte de eventos.
- Apache Beam: transformaciones del pipeline.
- KafkaIO: lectura y escritura de Kafka desde Beam.
- Expansion Service: soporte de transforms KafkaIO Java desde Python.
- Docker Compose: orquestación del entorno.

2. Arquitectura técnica

Servicio	Imagen	Función	Puerto
kafka	apache/kafka:4.0.1	Broker Kafka en KRaft	9094 externo / 9092 interno
expansion	apache/beam_java_expansion_service:2.69.0	Expansion Service	45193
beam	fpuna-logistream-beam	Pipeline Python / DirectRunner	—

Dentro de Docker, Beam utiliza kafka:9092. El productor ejecutado desde Windows utiliza localhost:9094. El Expansion Service se expone en el puerto 45193.

2.1 Tópicos Kafka

Tópico	Tipo	Propósito
logistics.events.v1	Entrada	Eventos logísticos JSON
logistics.invalid.v1	Salida	Eventos rechazados y motivo
logistics.metrics.v1	Salida	Métricas por ruta y ventana

3. Contrato de eventos

El pipeline espera event_id, event_time y payload. El payload debe contener vehicle_id, route_id, package_id, status y delay_minutes.

```
{
  "event_id": "evt-0001",
  "event_time": "2026-10-01T00:51:22Z",
  "payload": {
    "vehicle_id": "VEH-01",
    "route_id": "RUTA-01",
    "package_id": "PKG-001",
    "status": "DELIVERED",
    "delay_minutes": 5
  }
}
```

3.1 Clave de negocio

route_id se utiliza como clave en la salida de métricas, manteniendo juntas las métricas asociadas a una misma ruta.

4. Procesamiento Apache Beam

1. Lectura: ReadFromKafka consume logistics.events.v1 con group.id logistream-beam y auto.offset.reset=earliest.

2. Decodificación: El value de Kafka se convierte de bytes a JSON.

3. Validación: Se verifican campos obligatorios y la validez de event_time.

4. Separación: Los inválidos se envían a logistics.invalid.v1.

5. Event Time: event_time se asigna como TimestampedValue de Beam.

6. Deduplicación: event_id repetido no continúa hacia la agregación.

7. Ventana: FixedWindows de 60 segundos.

8. Lateness: Se permiten 120 segundos de atraso.

9. Triggers: Early cada 10 s de processing time, on-time por watermark y late por elemento.

10. Agrupación: Los eventos se agrupan por route_id dentro de cada ventana.

11. Métrica: Se calcula event_count, promedio de delay_minutes y event_ids.

12. Salida: Las métricas se publican en logistics.metrics.v1.

5. Tiempo de evento y ventanas

Parámetro	Configuración
Tipo	FixedWindows
Duración	60 segundos
Tiempo utilizado	event_time
Allowed lateness	120 segundos
Early trigger	Cada 10 segundos
On-time	Watermark supera el final de la ventana

Late trigger	Cada elemento tardío
Acumulación	ACCUMULATING

La estrategia permite procesar eventos que llegan fuera de orden y aceptar eventos tardíos mientras permanezcan dentro del límite definido.

6. Deduplicación e idempotencia

La implementación mantiene un conjunto de event_id observados durante la ejecución del worker. Si el mismo identificador aparece nuevamente, no participa nuevamente en la agregación.

```
evt-0003 → procesado
evt-0003 → duplicado → descartado de la agregación
```

La métrica posee un metric_id determinístico basado en route_id y el inicio de la ventana. Las salidas Kafka se configuran con enable.idempotence=true y acks=all.

No se declara exactly-once end-to-end; la deduplicación implementada corresponde a la estrategia de la demostración local.

7. Formato de métrica

```
{
  "metric_id": "RUTA-01:...",
  "window_start": "2026-10-01T00:51:00Z",
  "window_end": "2026-10-01T00:52:00Z",
  "route_id": "RUTA-01",
  "event_count": 3,
  "average_delay_minutes": 4.33,
  "event_ids": ["evt-0001", "evt-0003", "evt-0005"]
}
```

- metric_id: identificador determinístico de ruta + ventana.
- window_start/window_end: límites temporales.
- event_count: cantidad de eventos.
- average_delay_minutes: promedio del atraso.
- event_ids: eventos incluidos en la métrica.

8. Escenarios de prueba

Escenario	Eventos	Objetivo
Normal	evt-0001, evt-0002, evt-0003	Flujo básico
Duplicado	evt-0003 repetido	Deduplicación
Fuera de orden	evt-0004 y evt-0005	Event time vs. orden de llegada
Tardío	evt-0006	Allowed lateness
Muy tardío	evt-0007	Comportamiento ante gran atraso

El productor fue ejecutado y mostró correctamente en consola el envío de los escenarios anteriores.

9. Instalación y ejecución

9.1 Construcción

```
docker compose build beam
```

9.2 Infraestructura

```
docker compose up -d kafka expansion
```

9.3 Pipeline

```
docker compose up beam
```

9.4 Productor

```
python producer\producer.py
```

9.5 Ver tópicos

```
docker exec logistream-kafka /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

9.6 Consumir métricas

```
docker exec -it logistream-kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
localhost:9092 --topic logistics.metrics.v1 --from-beginning
```

9.7 Consumir inválidos

```
docker exec -it logistream-kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
localhost:9092 --topic logistics.invalid.v1 --from-beginning
```

9.8 Detener

```
docker compose down
```

10. Verificaciones realizadas

Componente	Estado	Evidencia
Kafka	OK	Contenedor levantado y tópicos creados.
KafkaIO	OK	Importación ReadFromKafka/WriteToKafka validada.
pipeline.py	OK	py_compile ejecutado sin errores.
Java	OK	OpenJDK 21 disponible en la imagen Beam.
Expansion Service	OK	Servicio 2.69.0 operativo en 45193 durante las pruebas.
Productor	OK	Eventos publicados en logistics.events.v1.
Docker desde Beam	OK	docker --version y hello-world ejecutados.
End-to-end	Pendiente	Persistió una incidencia con el worker Java/DirectRunner.

11. Incidencia conocida

Durante la integración final apareció una incidencia al inicializar el worker Java de Apache Beam. Se observó el mensaje 'Failed to obtain provisioning information: failed to dial server at localhost:43795' y posteriormente workers apache/beam_java11_sdk:2.69.0 que terminaron con código 1.

Se corrigió la disponibilidad del cliente Docker dentro de la imagen Beam y se verificó que Docker funciona desde el contenedor. También se verificó que el Expansion Service puede permanecer operativo.

La incidencia afecta la demostración completa fuente → Kafka → Beam → salida en este entorno local. No modifica la implementación documentada del contrato, validación, event time, ventanas, deduplicación y agregación.

11.1 Recomendación

Para una siguiente iteración se recomienda aislar el entorno de workers de Beam o utilizar un runner/entorno de ejecución compatible con el modo de despliegue elegido y repetir el smoke test end-to-end.

12. Limitaciones y mejoras futuras

- Reemplazar el set local de deduplicación por estado administrado de Beam con expiración.
- Agregar TestPipeline y TestStream para pruebas de event time, ventanas y lateness.
- Agregar evidencia automatizada del contenido de logistics.metrics.v1.
- Evaluar un runner de streaming apropiado para un despliegue más cercano a producción.
- Fijar versiones de dependencias y documentar compatibilidad.

13. Conclusión

LogiStream implementa una arquitectura de streaming basada en Kafka y Apache Beam con contrato de datos, validación, separación de inválidos, event time, deduplicación, ventanas fijas, allowed lateness, triggers, agregación por ruta y publicación de métricas.

La infraestructura Docker y el productor fueron validados. Al cierre de la entrega, la única limitación identificada corresponde a la inicialización del worker Java de Apache Beam durante la ejecución end-to-end local.